IN403 · Advanced Network Protocols, Architectures & Security

LECTURE 02 · 2 HOURS

TCP Congestion Control I: Tahoe, Reno & NewReno

Why networks collapse, how TCP probes for capacity, and the loss-based AIMD algorithms that made the Internet stable



Course topic: TCP/IP Deep Dive

Learning objectives

By the end of this session you will be able to:

1. Explain congestion collapse and distinguish congestion control from flow control
2. Describe slow start, congestion avoidance and the role of cwnd and ssthresh
3. Trace Tahoe, Reno and NewReno through timeout and fast-retransmit events
4. Explain RTO estimation (SRTT, RTTVAR, Karn’s algorithm) and SACK-based recovery
5. Reason about AIMD fairness and the Mathis throughput model

Session plan (120 minutes)



0:00 – 0:10	Recap of Lecture 1 and motivation: congestion collapse
0:10 – 0:35	cwnd, slow start and congestion avoidance
0:35 – 1:00	Loss detection: RTO, duplicate ACKs, fast retransmit
1:00 – 1:10	Break
1:10 – 1:35	Tahoe vs Reno vs NewReno, SACK recovery
1:35 – 1:50	Activity: trace a cwnd evolution
1:50 – 2:00	AIMD fairness, quiz and wrap-up

01

Why congestion control?

From the 1986 congestion collapse to Van Jacobson’s algorithms

Congestion collapse

- October 1986: throughput between LBL and UC Berkeley dropped from 32 kb/s to 40 b/s — a factor of ~1000
- Cause: senders retransmitted aggressively into already-full queues; most delivered packets were duplicates
- Van Jacobson (1988) added slow start, congestion avoidance, better RTT estimation and fast retransmit to BSD TCP
- Congestion symptoms
  - Long queueing delays in routers
  - Packet loss when buffers overflow
  - Wasted upstream capacity on packets later dropped

?

Flow vs congestion control

Flow control protects the receiver (rwnd, advertised explicitly). Congestion control protects the network (cwnd, inferred by the sender from loss or delay signals). The sender obeys whichever is smaller.

REMEMBER

Congestion control is a cooperative, distributed algorithm running in every sender. It protects the shared network, not the receiver.

Congestion collapse

EXPLAINED

A network is congested when packets arrive at a router faster than its output link can send them. The router queues them, and when the queue is full it must drop new arrivals. Delay rises first, then loss.

Early TCP had flow control but no congestion control. When packets were lost, senders simply retransmitted the whole window — into the same full queues. Most of the capacity was spent carrying copies that would be dropped further along the path or that duplicated data already delivered. Useful throughput collapsed to almost nothing even though every link was 100% busy.

Van Jacobson’s fix was to make every sender slow down when it detects loss and speed up carefully when it does not. Because every TCP host follows the same rules, the network as a whole stays stable without any central controller — one of the most important design ideas in the Internet.

EXAMPLE

A road analogy: if every car that misses a junction immediately turns round and tries again, the junction jams completely. Congestion control is like every driver agreeing to wait longer each time they fail.

Two state variables drive everything

C

cwnd

Congestion window, in bytes or segments. Limits unacknowledged data the sender may have in flight. Kept privately by the sender.

S

ssthresh

Slow-start threshold. Below it, cwnd grows exponentially; above it, linearly. Set to half the flight size when loss is detected.

E

Effective window

EffectiveWindow = min(cwnd, rwnd) − bytes in flight. The connection sends only when this is positive.

R

Rate ≈ cwnd / RTT

Throughput is controlled indirectly by adjusting the window: a self-clocking “ACK clock” paces new transmissions.

Two state variables drive everything

EXPLAINED

TCP congestion control is implemented with just two numbers kept by the sender. cwnd (congestion window) is the sender's estimate of how much data the network can absorb right now. It is never sent on the wire; it is the sender's private belief.

ssthresh (slow-start threshold) remembers roughly where trouble happened last time. Below ssthresh TCP grows cwnd quickly; above it TCP grows cautiously. After a loss, ssthresh is set to half the amount of data that was in flight.

The sender may only have min(cwnd, rwnd) bytes in flight. Because new segments are sent only when ACKs return, and ACKs return at the speed of the bottleneck link, TCP is self-clocking: the ACK stream automatically paces the sender at the right rate.

EXAMPLE

cwnd = 20 segments, rwnd = 50 segments the sender may have 20 segments in flight. If rwnd dropped to 10, the receiver would be the limit instead.

REMEMBER

Sending rate ≈ cwnd / RTT. Changing cwnd is how TCP speeds up or slows down.

02

Slow start and congestion avoidance

Probe quickly, then cautiously

The two growth phases

Slow start (cwnd < ssthresh)

- Start with IW = 10 MSS (RFC 6928)
- cwnd += 1 MSS for each ACK received
- Doubles every RTT — exponential growth
- Reaches a BDP of W segments in ~$\log_2 W$ RTTs
- Ends at ssthresh or on the first loss

Congestion avoidance (cwnd ≥ ssthresh)

- cwnd += MSS × MSS / cwnd per ACK
- About +1 MSS per RTT — linear growth
- Additive Increase of AIMD
- Gently probes for spare capacity
- Produces the classic “saw-tooth”

EXAMPLE

IW = 10, ssthresh = 80: cwnd goes 10 ! 20 ! 40 ! 80 in three RTTs (slow start), then 81 ! 82 ! 83 … one per RTT (congestion avoidance).

REMEMBER

Slow start: ×2 per RTT. Congestion avoidance: +1 MSS per RTT. Loss: cut the window.

The two growth phases

EXPLAINED

A new connection has no idea how much capacity is available. Slow start finds out quickly: for every ACK received, cwnd increases by one segment. Since a full window of ACKs comes back each RTT, cwnd doubles every round trip — exponential growth.

Once cwnd reaches ssthresh, TCP switches to congestion avoidance. Now cwnd increases by about one segment per RTT (each ACK adds MSS × MSS / cwnd). This linear growth gently probes for extra capacity without causing big bursts of loss.

When loss eventually occurs, cwnd is cut (multiplicative decrease) and growth starts again. Plotted over time, cwnd forms the famous saw-tooth pattern: slow climb, sudden drop, slow climb.

Slow start in numbers (IW = 10, MSS = 1460 B)

RTT #	cwnd (segments)	cwnd (bytes)	Rate at 50 ms RTT
1	10	14.6 KB	2.3 Mb/s
2	20	29.2 KB	4.7 Mb/s
3	40	58.4 KB	9.3 Mb/s
4	80	117 KB	18.7 Mb/s
5	160	234 KB	37.4 Mb/s
6	320	467 KB	74.8 Mb/s
7	640	934 KB	150 Mb/s

Short transfers (most web objects) finish while still in slow start — their latency is dominated by RTT, not bandwidth.

Slow start in numbers (IW = 10, MSS = 1460 B)

EXPLAINED

This table shows how fast slow start really is. Starting from the modern initial window of 10 segments (about 14.6 KB), the rate doubles each round trip. After 7 RTTs — only 350 ms on a 50 ms path — the connection could already send 150 Mb/s.

The consequence is that short transfers never leave slow start. A typical web object of 50–100 KB completes in 3–4 round trips, long before TCP has discovered the link capacity. For such transfers, latency (number of RTTs) matters much more than bandwidth.

This is why the initial window was raised from 1–4 segments to 10 (RFC 6928), why connections are reused, and why later protocols such as QUIC try to remove handshake round trips.

EXAMPLE

A 100 KB page: RTT 1 sends 14.6 KB, RTT 2 another 29.2 KB (43.8 KB total), RTT 3 another 58.4 KB done in 3 RTTs, at most 9.3 Mb/s, even on a 1 Gb/s link.

REMEMBER

For small objects, reduce round trips; for large transfers, buffers and congestion control decide throughput.

03

Detecting loss

Retransmission timeouts and duplicate acknowledgments

Retransmission timeout (RFC 6298)

```
On first RTT sample R:
  SRTT   = R
  RTTVAR = R / 2

On each later sample R':
  RTTVAR = (1 - β)·RTTVAR + β·|SRTT - R'|   β = 1/4
  SRTT   = (1 - α)·SRTT   + α·R'            α = 1/8

RTO = SRTT + max(G, 4·RTTVAR)
RTO = max(RTO, 1 s)     # Linux uses 200 ms min

On timeout: RTO = 2·RTO  (exponential back-off)
```

- Karn’s algorithm: never take RTT samples from retransmitted segments (ambiguous)
- The Timestamps option removes this ambiguity and gives a sample per ACK
- A timeout is a strong congestion signal — the ACK clock is lost

Retransmission timeout (RFC 6298)

EXPLAINED

If an ACK does not arrive in time, the sender assumes the segment was lost and retransmits it. Choosing “in time” is hard: too short and we retransmit packets that were only delayed; too long and we waste time after a real loss.

TCP keeps a smoothed RTT (SRTT) — a moving average of measured round-trip times — and RTTVAR, a moving average of how much the RTT varies. The timeout is RTO = SRTT + 4 × RTTVAR, so it is slightly longer than a typical RTT on stable paths and much longer on jittery paths. Each consecutive timeout doubles the RTO (exponential back-off).

Karn’s algorithm adds one rule: never use the RTT of a retransmitted segment, because when its ACK arrives we cannot tell whether it acknowledges the original or the copy. A timeout is a strong signal: it means even the ACK clock has stopped, so TCP reacts severely.

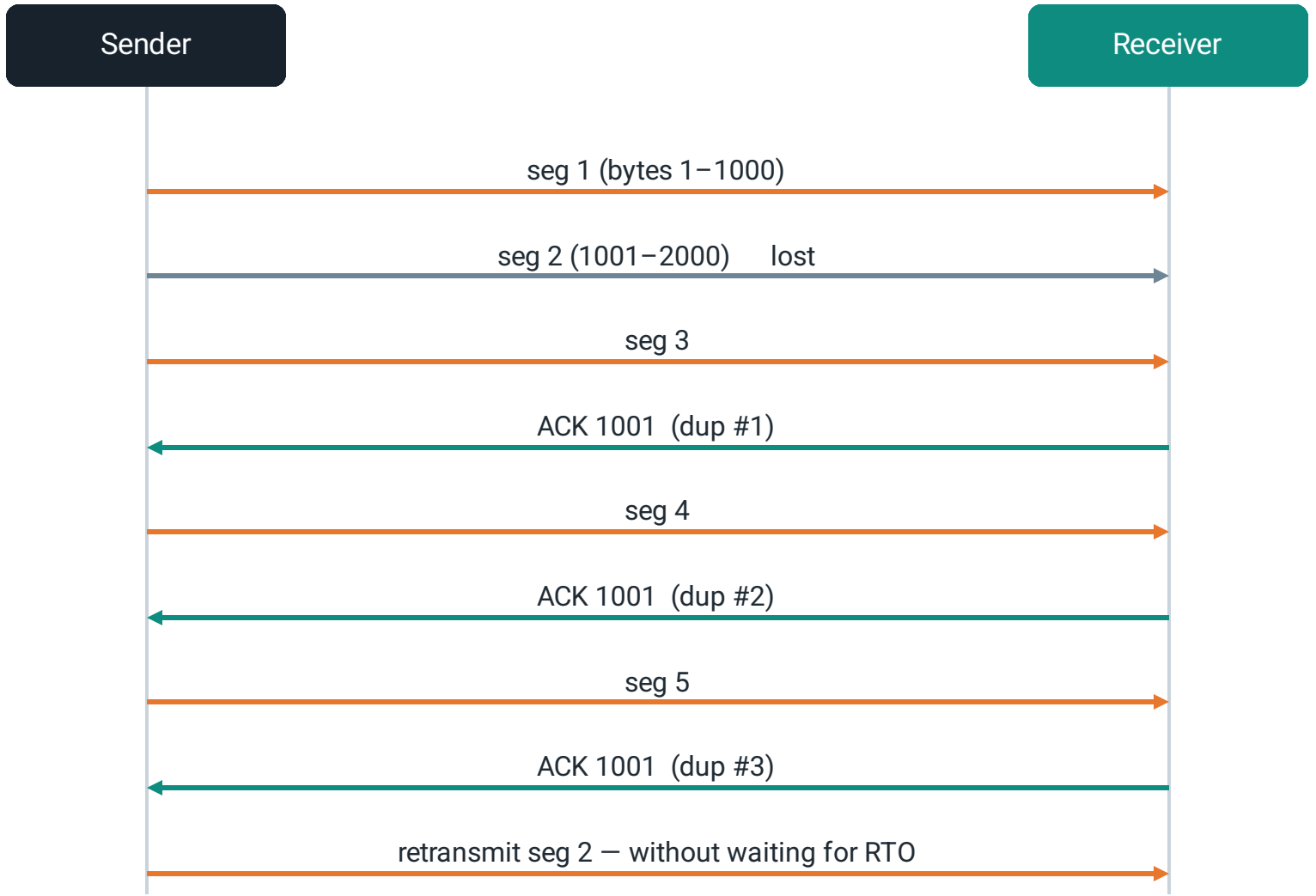
EXAMPLE

Samples 100, 120 ms: after the first, SRTT = 100, RTTVAR = 50. After the second: RTTVAR = 0.75×50 + 0.25×20 = 42.5; SRTT = 0.875×100 + 0.125×120 = 102.5 RTO = 102.5 + 170 = 272.5 ms (then floored to the minimum, e.g. 1 s by RFC, 200 ms on Linux).

REMEMBER

RTO adapts to both average RTT and its variability. A timeout resets cwnd to 1 MSS.

Fast retransmit: three duplicate ACKs



- Out-of-order arrival triggers an immediate duplicate ACK
- 3 dupACKs segment very likely lost, not just reordered
- Data is still flowing, so congestion is mild — no need to restart from cwnd = 1
- Saves a full RTO (often hundreds of ms)

Fast retransmit: three duplicate ACKs

EXPLAINED

Waiting for a full timeout after every loss would be slow. TCP has a faster signal. When a segment is missing but later ones arrive, the receiver cannot advance its cumulative ACK, so it repeats the same ACK number — a duplicate ACK — for every out-of-order segment.

One or two duplicates can be caused by simple reordering in the network. Three duplicate ACKs strongly suggest that a segment was really lost. The sender then retransmits the missing segment immediately, without waiting for the timer: this is fast retransmit.

Duplicate ACKs also carry good news: each one means a later segment has left the network and reached the receiver. Data is still flowing, so the congestion is probably mild — which is why TCP does not need to collapse cwnd to 1 here.

EXAMPLE

Segments 1–5 sent, segment 2 lost. The receiver ACKs 1001 after seg 1 and again after segs 3, 4 and 5. On the 3rd duplicate, the sender resends seg 2 — perhaps 300 ms earlier than an RTO would fire.

REMEMBER

3 dupACKs = mild congestion, fast retransmit. Timeout = severe congestion, back to slow start.

04

Tahoe, Reno and NewReno

Three generations of loss reaction

Comparing the classic variants

Event	Tahoe (1988)	Reno (1990)	NewReno (RFC 6582)
Timeout	ssthresh = cwnd/2; cwnd = 1; slow start	Same as Tahoe	Same as Tahoe
3 dup ACKs	Fast retransmit, then cwnd = 1 (slow start)	ssthresh = cwnd/2; cwnd = ssthresh + 3; fast recovery	Same entry as Reno
During recovery	—	cwnd += 1 per extra dupACK (inflation)	Same inflation
Partial ACK	—	Exits recovery; further losses often need RTO	Stays in recovery, retransmits next hole immediately
Multiple losses / window	Poor	Poor — often times out	One hole per RTT, no timeout

Comparing the classic variants

EXPLAINED

Tahoe (1988) treats every loss the same way: set ssthresh to half, reset cwnd to 1 and slow start again. It is safe but wastes capacity after each loss.

Reno (1990) adds fast recovery: after three duplicate ACKs it halves cwnd instead of resetting it, and continues sending new data while the missing segment is repaired. This works well for one loss per window, but if several segments are lost from the same window Reno often exits recovery too early and ends up waiting for a timeout anyway.

NewReno fixes this with the idea of a partial ACK: an ACK that covers the first retransmission but not all data that was outstanding when recovery started. It means another segment is missing, so NewReno retransmits it immediately and stays in recovery. Each RTT repairs one hole without any timeout.

EXAMPLE

Window of 20 segments, segments 5 and 9 lost. Reno retransmits 5, then the ACK for 5 jumps only to 9 Reno leaves recovery and later times out. NewReno sees the partial ACK and retransmits 9 straight away.

REMEMBER

Tahoe: always restart. Reno: halve and recover one loss. NewReno: recover many losses, one per RTT.

Reno fast recovery step by step



3 dup ACKs

Set ssthresh = max(FlightSize/2, 2 MSS). Retransmit the missing segment.

Inflate

cwnd = ssthresh + 3 MSS, accounting for the three segments that left the network.

Keep sending

Each further dupACK adds 1 MSS to cwnd, so new data can be sent while waiting.

Deflate

When the new ACK arrives, set cwnd = ssthresh and enter congestion avoidance.

Reno fast recovery step by step

EXPLAINED

Step 1: on the third duplicate ACK, the sender sets ssthresh to half of the data in flight and retransmits the missing segment. Step 2: it sets cwnd to ssthresh + 3, because the three duplicate ACKs prove that three segments have left the network.

Step 3: every additional duplicate ACK means one more segment has been delivered, so cwnd is temporarily inflated by one segment, allowing a new segment to be sent. This keeps the ACK clock running during recovery.

Step 4: when an ACK for new data finally arrives (the retransmission succeeded), cwnd is deflated back to ssthresh and the sender continues in congestion avoidance. Overall the window has been halved — the “multiplicative decrease” of AIMD — but the pipe never emptied.

EXAMPLE

cwnd = 40 when 3 dupACKs arrive ssthresh = 20, cwnd = 23. Ten more dupACKs cwnd = 33 (new segments sent). New ACK cwnd = 20, then +1 per RTT.

REMEMBER

Fast recovery keeps data flowing during repair, then resumes from half the previous window.

SACK-based loss recovery

- Problem: cumulative ACKs reveal only the first hole; multiple losses take multiple RTTs
- SACK blocks (up to 3–4 per ACK) tell the sender exactly which ranges arrived
- Scoreboard: the sender tracks SACKed, lost and in-flight bytes
- Pipe algorithm (RFC 6675): send while estimated “pipe” < cwnd, filling holes first
- Modern Linux: RACK-TLP (RFC 8985) detects loss by time, not dupACK counts, handling reordering and tail losses better

S

In practice

Virtually all modern stacks negotiate SACK. Wireshark shows SACK blocks as “SLE/SRE” (left/right edges) inside the TCP options of duplicate ACKs.

SACK-based loss recovery

EXPLAINED

Cumulative ACKs only say “I have everything up to byte N”. If segments 5, 9 and 14 are lost, the sender learns about them one at a time, one per RTT. With Selective Acknowledgment, the receiver also lists the blocks it has received beyond the hole, e.g. “I have 10–13 and 15–20”.

The sender keeps a scoreboard of which bytes are SACKed, which are presumed lost and which are still in flight. The pipe algorithm then retransmits all the holes as quickly as the congestion window allows, instead of one per RTT.

Modern Linux goes further with RACK-TLP: rather than counting duplicate ACKs, it declares a segment lost if a segment sent later has already been delivered and enough time has passed. This handles reordering better and also detects losses at the end of a transfer (tail losses) where there are no later segments to generate duplicate ACKs.

EXAMPLE

ACK = 5001, SACK blocks 6001–9000 and 10001–12000 the sender knows exactly two ranges are missing: 5001–6000 and 9001–10000, and resends both in the same RTT.

REMEMBER

SACK tells the sender exactly what is missing, so multiple losses are repaired in one round trip.

05

AIMD and fairness

Why additive increase + multiplicative decrease converges

AIMD converges to a fair share

The Chiu–Jain argument

- Two flows share capacity C
- Additive increase moves both rates up by the same amount (45° line)
- Multiplicative decrease halves both — moves toward the origin
- Repeated cycles converge to the fairness line $x_1 = x_2$

Caveats

- Fair per flow, not per user — parallel connections grab more
- RTT unfairness: short-RTT flows grow faster and win
- UDP flows without congestion control are not constrained
- Loss-based TCP fills buffers bufferbloat

AIMD converges to a fair share

EXPLAINED

Imagine two flows sharing a link. Draw flow 1’s rate on one axis and flow 2’s on the other. Additive increase moves the point diagonally (both gain the same amount). Multiplicative decrease moves it towards the origin (both lose the same fraction), which shrinks the gap between them in absolute terms.

Repeating increase and decrease many times pushes the point towards the line where both rates are equal: AIMD converges to fairness and efficiency (using the capacity). Other combinations, like additive decrease, do not converge.

In practice fairness has limits. TCP is fair per connection, so an application that opens ten connections gets about ten shares. Flows with shorter RTTs receive ACKs sooner and grow faster, so they get more bandwidth (RTT unfairness). And because loss-based TCP keeps increasing until buffers overflow, it keeps queues full, causing high delay (bufferbloat).

EXAMPLE

Flows at 80 and 20 Mb/s on a 100 Mb/s link: after loss both halve 40 and 10 (gap 30). Both add equally 65 and 35 at the next loss (gap still 30), halve 32.5 and 17.5 (gap 15). The gap keeps shrinking.

REMEMBER

AIMD is fair between similar flows, but favours short-RTT flows and applications with many connections.

The Mathis model: throughput vs loss

≈ 1.22

Constant √(3/2) for Reno

MSS/RTT

Scales with segment size and inversely with RTT

1/√p

Throughput falls with the square root of loss rate p

Throughput ≈ (MSS / RTT) × 1.22 / √p. Example: MSS 1460 B, RTT 100 ms, p = 10^{-4} ≈ 14 Mb/s. To reach 10 Gb/s at 100 ms you would need p ≈ 2×10^{-10} — one loss every ~5 billion packets. This motivates CUBIC and BBR (Lecture 3).

The Mathis model: throughput vs loss

EXPLAINED

The Mathis formula estimates the average throughput of a long-running Reno flow from three quantities: segment size, round-trip time and loss probability. Throughput ≈ (MSS / RTT) × 1.22 / √p.

It comes from the saw-tooth: the window oscillates between W/2 and W, so on average it is 3W/4, and one loss happens per cycle. Working out how many packets one cycle contains gives the 1/√p dependency.

The formula explains several practical facts: throughput falls as RTT grows (long-distance transfers are slower), it falls with the square root of loss (1% loss is very damaging) and, most importantly, reaching very high speeds over long paths requires unrealistically low loss rates. This is the motivation for the high-speed algorithms in the next lecture.

EXAMPLE

MSS 1460 B, RTT 100 ms, p = 0.01% (11 680 b / 0.1 s) × 1.22 / 0.01 ≈ 14.2 Mb/s. With 1% loss: ≈ 1.4 Mb/s.

REMEMBER

Reno throughput ∀MSS / (RTT × √p). Long, fast paths need algorithms that do not depend so heavily on loss.

Trace the congestion window

IN-CLASS ACTIVITY · 15 MIN

Reno sender, MSS units, IW = 1, initial ssthresh = 16. Plot cwnd for RTTs 1–14.

- Slow start until ssthresh, then congestion avoidance
- At the end of RTT 8, three duplicate ACKs are received
- At the end of RTT 12, a retransmission timeout occurs
- Mark the slow-start and congestion-avoidance regions and every ssthresh value
- Bonus: repeat for Tahoe and compare the area under the curves

Check your understanding

Q1 Why does a retransmission timeout reduce cwnd to 1 MSS while three duplicate ACKs only halve it?

Q2 What problem does NewReno solve that Reno cannot, and what is a “partial ACK”?

Q3 Why does Karn’s algorithm ignore RTT samples from retransmitted segments?

Q4 Two Reno flows share a bottleneck; one has 20 ms RTT and the other 200 ms. Which gets more throughput and why?

Model answers

A1 A timeout means no ACKs are returning at all — the ACK clock has stopped and congestion may be severe, so TCP restarts carefully. Three dupACKs show later segments are still arriving, so the network is still delivering and halving is enough.

A2 Reno handles only one loss per window well; with several losses it leaves recovery too early and often times out. A partial ACK acknowledges the retransmitted segment but not everything outstanding; NewReno uses it to retransmit the next hole immediately.

A3 When the ACK arrives you cannot know whether it acknowledges the original transmission or the retransmission, so the RTT sample could be wildly wrong and corrupt SRTT/RTO.

A4 The 20 ms flow gets ACKs ten times as often, so it increases cwnd ten times faster in congestion avoidance and recovers faster after loss: it takes most of the bandwidth (RTT unfairness).

Key takeaways

- Congestion control protects the network; the sender limits itself to min(cwnd, rwnd)
- Slow start doubles cwnd per RTT; congestion avoidance adds ~1 MSS per RTT
- Timeouts are severe signals (cwnd ! 1); three dupACKs trigger fast retransmit and recovery
- NewReno and SACK recover from multiple losses per window without timeouts
- AIMD converges to fairness, but Reno scales poorly on high-BDP paths (Mathis: ∀1/√p)

Reading & next session: Kurose & Ross §3.6–3.7 · RFC 5681, 6298, 6582, 6675 · Next: Lecture 3 — CUBIC, BBR & TCP Performance Optimization